COMPUTING 2 - PARALLEL COMPUTING

Chapter 5 Mock Test

Tiling, Matrix Multiplication, and Occupancy

Suggested time	Total points	Coverage
55 minutes	60	Global-memory request counts, tiled GEMM, operational intensity, banks, resources

NAME		STUDENT ID	
DATE		SCORE	/ 60

Instructions
- Answer every question and show essential reasoning for partial credit.
- For true/false items, an unsupported letter receives no credit.
- Unless stated otherwise, assume NVIDIA warp size 32 and ideal arithmetic.
- Keep the answer section closed until the suggested time has elapsed.

Questions

5.1 Per-element global-memory requests — 8 points

For C=AB with square N by N matrices, how many times is each input element requested from global memory (including caches) (a) without tiling and (b) with T by T shared-memory tiles? State the assumption behind the tiled count.

WORK AREA

5.2 Total load count — 8 points

Use N=4096 and T=32. Compute naive and tiled input-load counts for the full multiplication, the reduction factor, and per-element request counts.

WORK AREA

5.3 Tiled GEMM operational intensity — 8 points

For one phase of one T by T output tile, a block loads one T by T tile from A and one from B, then performs $2T^3$ FLOPs. Ignoring output traffic, derive operational intensity for FP32 and evaluate T=32.

WORK AREA

5.4 Complete a tiled GEMM phase — 10 points

Fill the missing synchronization points and bounds behavior. Explain why two barriers are commonly needed per phase.

```
for (int ph = 0; ph < numTiles; ++ph) {
    As[ty][tx] = /* guarded A load or zero */;
    Bs[ty][tx] = /* guarded B load or zero */;
    // missing
    for (int k = 0; k < T; ++k) acc += As[ty][k] * Bs[k][tx];
    // missing
}
/* guarded C store */
```

WORK AREA

5.5 Shared-memory bank conflict — 7 points

A 32 by 32 float tile is stored row-major in 32 shared-memory banks. A warp reads tile[threadIdx.x][fixedCol]. Explain the conflict and the classic padding fix.

WORK AREA

5.6 Occupancy with three limits — 10 points

An SM supports 2048 threads, 65,536 registers, 96 KiB shared memory, and 32 blocks. A 256-thread block uses 48 registers/thread and 24 KiB shared memory. Find resident blocks and thread occupancy.

WORK AREA

5.7 Registers versus shared memory — 9 points

Assume capacity is not a problem. Give one important reason to place a globally loaded value in shared memory rather than in a register. Also give one reason to prefer a register.

WORK AREA

ANSWER SECTION

Chapter 5: Answer Key & Worked Solutions

Grade the method as well as the final result. Numerical answers show enough intermediate work to identify where an error occurred.

Quick Answer Key

Q	Topic	Short answer
5.1	Per-element global-memory requests	N times without tiling; N/T times with tiling
5.2	Total load count	naive 2N^3=137,438,953,472; tiled 2N^3/T=4,294,967,296; 32x; 4096 vs 128
5.3	Tiled GEMM operational intensity	I=T/4 FLOP/byte; at T=32, I=8 FLOP/byte
5.4	Complete a tiled GEMM phase	Barrier after load and before overwrite; zero-fill out-of-bounds; guarded store
5.5	Shared-memory bank conflict	Stride 32 maps all lanes to one bank; declare tile[32][33] to change the stride
5.6	Occupancy with three limits	4 blocks; 1024 resident threads; 50% occupancy
5.7	Registers versus shared memory	Shared permits cross-thread reuse and indexed access; registers are lowest-latency private storage

Worked Solutions

5.1 Per-element global-memory requests — 8 points

Answer. One A element is reused across N output columns, and one B element across N output rows.

- Naive: every consuming output thread requests the input element, so N requests per input element.
- Tiled: one tile load serves T output threads along the reuse dimension. There are N/T output tiles, so N/T requests per input element.
- Assume N is divisible by T and each block stages each needed tile once.

$$\text{request reduction factor} = N / (N/T) = T$$

5.2 Total load count — 8 points

Answer. There are N^2 outputs. Each naive output requests N values from A and N from B.

- Naive total=$2N^3$=137,438,953,472 input requests.
- Tiled total=$2N^3/T$=4,294,967,296.
- Reduction=32x.
- Each input element: 4096 requests naive and 4096/32=128 tiled.

Common trap. These are logical requests under the question's model; physical DRAM transactions may be reduced by caches.

5.3 Tiled GEMM operational intensity — 8 points

Answer. The two input tiles contain $2T^2$ floats, or $8T^2$ bytes.

- FLOPs=$2T^3$.
- Bytes=$2T^2$x4=$8T^2$.
- I=$(2T^3)/(8T^2)$=T/4 FLOP/byte.
- At T=32, I=8 FLOP/byte.

$$I = \text{FLOPs/bytes} = T/4 \text{ for idealized FP32 tiled GEMM}$$

5.4 Complete a tiled GEMM phase — 10 points

Answer. All threads must finish staging before consumers read, and all consumers must finish before any thread overwrites shared storage for the next phase.

```
for (int ph = 0; ph < numTiles; ++ph) {
    As[ty][tx] = (row < M && ph*T+tx < K) ? A[row*K + ph*T+tx] : 0.0f;
    Bs[ty][tx] = (ph*T+ty < K && col < N) ? B[(ph*T+ty)*N + col] : 0.0f;
    __syncthreads();
    for (int k = 0; k < T; ++k) acc += As[ty][k] * Bs[k][tx];
    __syncthreads();
}
if (row < M && col < N) C[row*N + col] = acc;
```

Common trap. Every thread in the block must reach both barriers, including threads whose global indices are out of range.

5.5 Shared-memory bank conflict — 7 points

Answer. Successive lanes read rows separated by 32 floats. With 32 banks, every address has the same bank index.

- The access becomes a 32-way bank conflict under the simplified bank model.
- Padding the second dimension to 33 changes row stride from 32 to 33 floats.
- Successive lanes then map to successive banks.

```
__shared__ float tile[32][33];
```

5.6 Occupancy with three limits — 10 points

Answer. Compute each limit in blocks and take the minimum.

- Threads: floor(2048/256)=8 blocks.
- Registers: floor(65536/(256x48))=floor(5.333)=5 blocks.
- Shared memory: floor(96/24)=4 blocks.
- Block-count limit=32. Minimum=4 blocks.
- Occupancy=(4x256)/2048=50%.

Common trap. High occupancy can hide latency, but the maximum-occupancy kernel is not automatically the fastest kernel.

5.7 Registers versus shared memory — 9 points

Answer. The decisive distinction is visibility and access pattern, not only latency.

- Shared memory lets multiple threads in the same block reuse data loaded by one or a subset of threads and supports indexed exchange such as transpose tiles.
- Registers are private to one thread and are ideal for accumulators and repeatedly used thread-local values.
- Register pressure can reduce occupancy or cause spills; shared memory also consumes a block-level resource and requires synchronization.